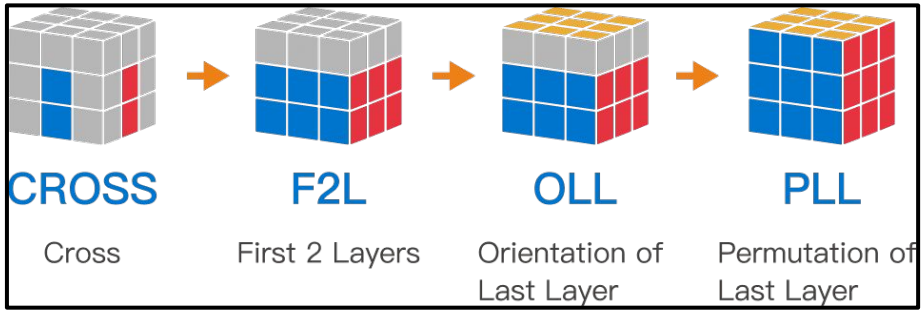
CS 4824 - Rubik's Cube Oracle Estimation

Caleb Fox, Ethan Avery

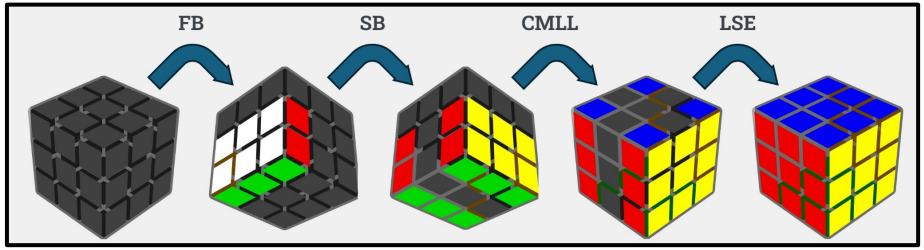
What is a Rubik's Cube method?

- Ordering of pieces to solve a cube
- Broken down into ordered steps
- Within a step, pieces can be solved in any way

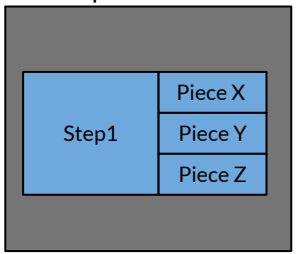
CFOP - (57.5 moves)



Roux - (48 moves)



Step breakdown



Oracle purpose

Search for better human-solvable methods

- To find better methods, tons need to be evaluated
- Evaluating classically is expensive, while evaluating with ML is fast

Related works:

- Using ML to solve the cube (DeepCubeA has done this)
- Surrogate Models and Ensemble Strategies for Expensive Evolutionary Optimization: An Industrial Case Study (link)

ML Oracle

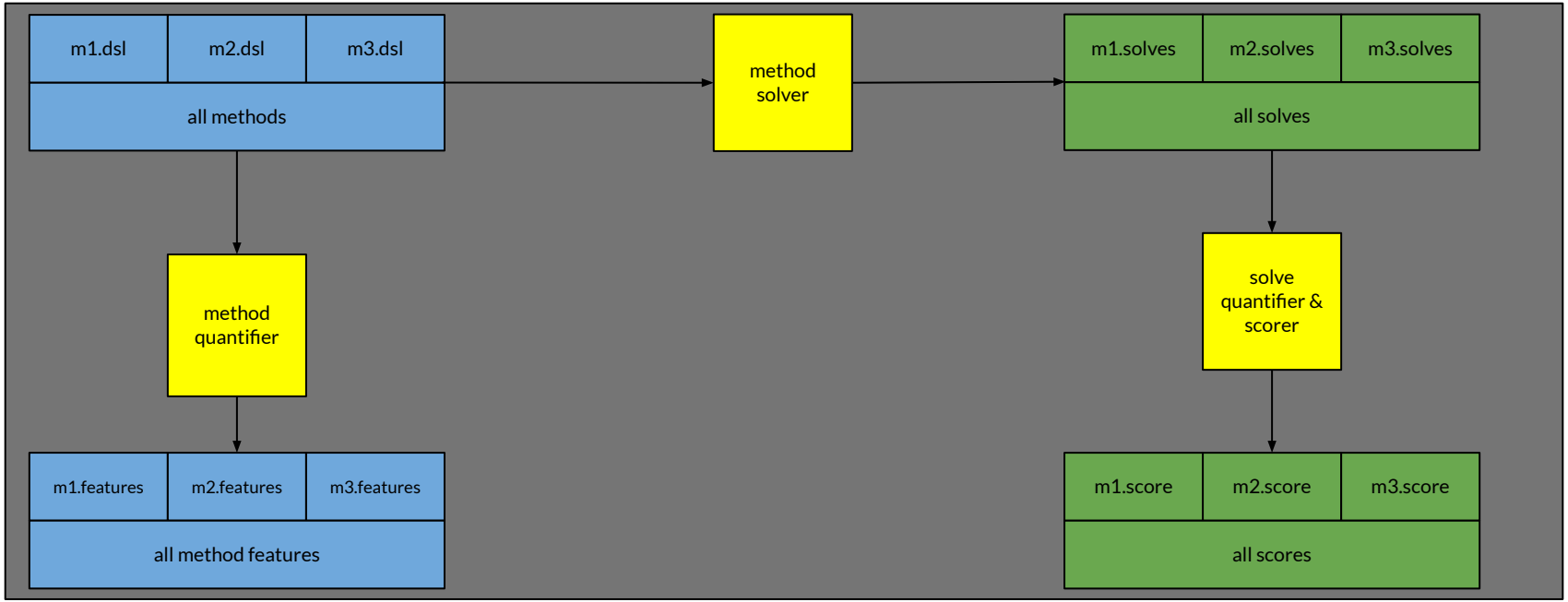
Classically:

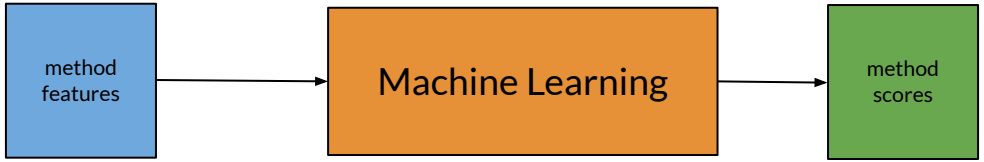
- One method: ~10 solves
- One solve: ~ 100 sec
- Time per single method: 100 * 10 = 1000 sec
- Evaluate 10k methods: 10,000 * 1000 = 1 * 10^7 secs =**115.7 days**

ML oracle:

- Evaluate 10k methods: **~9 minutes**

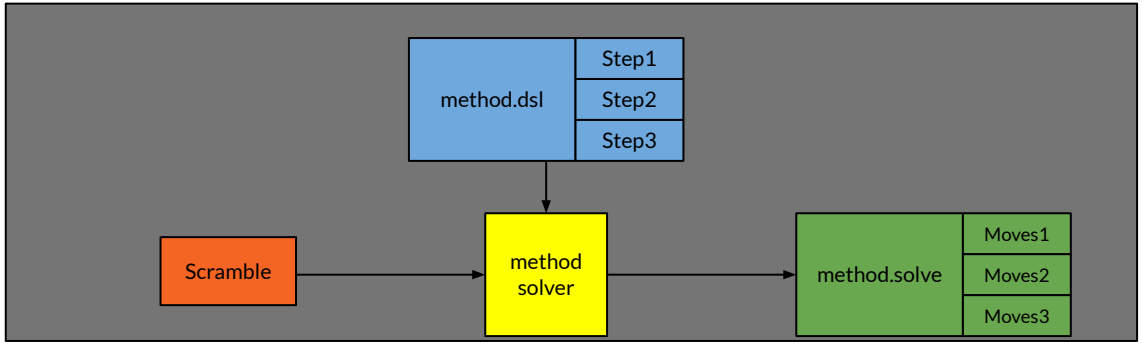
High level overview



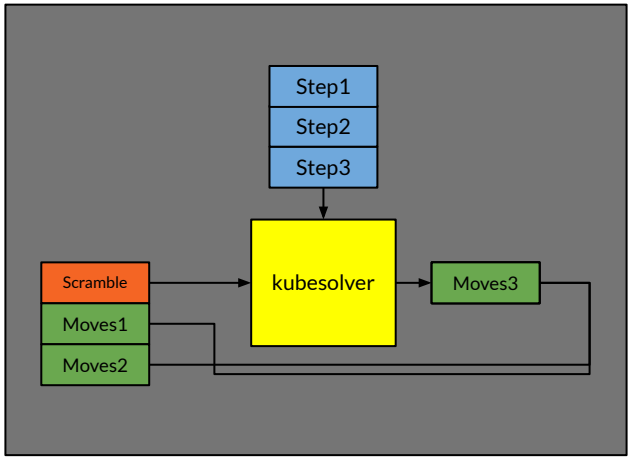


Method Solver

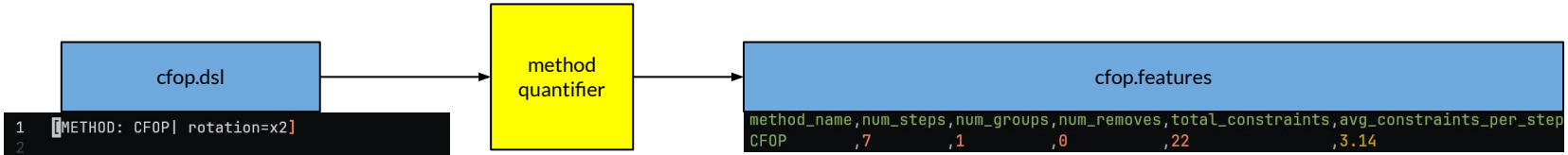
High Level Solver



Method Solver



Method Quantifier




```
[METHOD: CFOP| rotation=x2]

[STEP: cross]
init_empty_cube
add_edge UF
add_edge UB
add_edge UR
add_edge UL

[GROUP: F2L | order=best_1]
  [STEP: pair_fr]
  add_edge BR
  add_corner UBR
  [STEP: pair_fl]
  add_edge BL
  add_corner UBL
  [STEP: pair_bl]
  add_edge FL
  add_corner UFL
  [STEP: pair_br]
  add_edge FR
  add_corner UFR
[END GROUP]

[STEP: oll | cache_alg=true | free_layer=D]
add_corners_orientation
add_edges_orientation

[STEP: pll | cache_alg=true | free_layer=D]
add_edge FD
add_edge BD
add_edge LD
add_edge RD
add_corner DFR
add_corner DBR
add_corner DBL
add_corner DFL

[END METHOD]
```

Solve Quantifier

cfop.solves

```
scramble,orientation,cross,pair_fr,pair_bl,pair_fl,pair_br,oll,pll,total_moves

D U2 R D' F B D L2 F R' U R' U' D2 B2 R L2 B' R' F2 U' R2 B,
x2,
B2 D' L B' D,
U F' U2 F,
B' L2 B2 L B' L B,
R' F U F' R,
R' U2 F' U2 F R,
F L' U2 L U2 L F2 L' F,
L R U2 R' U' R U2 L' U R' U',
47
```

Solve quantifier

cfop.solves_features

```
method,total_solves,avg_total_moves,num_steps,avg_moves_per_step,num_algs,
CFOP   ,10          ,54.3           ,7         ,7.75              ,78
```

cfop.solves_features

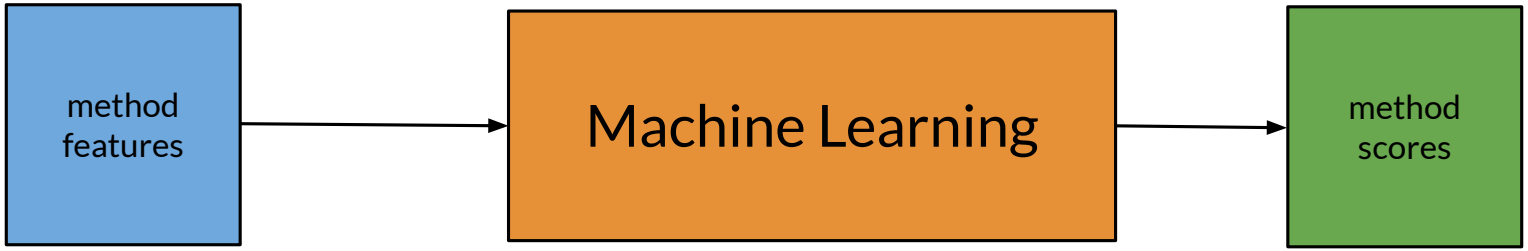
method scorer

```python
def score_method(eval_row: dict) -> float:
    avg_total_moves = float(eval_row.get("avg_total_moves", 0))
    avg_moves_per_step = float(eval_row.get("avg_moves_per_step", 0))
    total = int(eval_row.get("total_solves", 0))
    expected = NUM_SCRAMBLES

    if avg_total_moves <= 0 or total == 0:
        return 0.0

    completion = min(total / expected, 1.0)
    base_score = 1 / avg_total_moves

    # Human-practicality penalty:
    # no penalty up to 7 avg moves/step, then quadratic growth
    threshold = 7.0
    penalty_strength = 0.15

    if avg_moves_per_step <= threshold:
        step_penalty = 1.0
    else:
        excess = avg_moves_per_step - threshold
        step_penalty = 1 / (1 + penalty_strength * (excess ** 2))

    return round(base_score * (completion ** 2) * step_penalty, 6)
```

cfop.score

```
,score
,0.018936
```

Predicting the score



Features:

- First step is to extract features from data
- These describe complexity, structure, and constraints
- Features eventually scaled

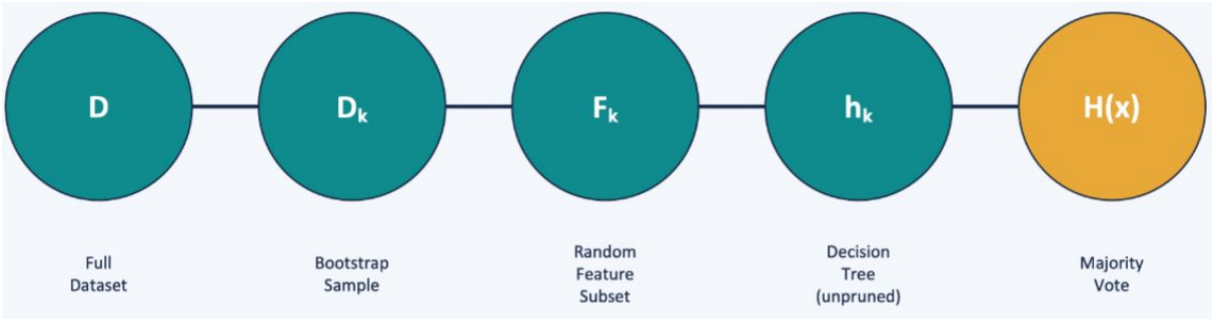
- num_steps
- num_groups
- num_removes
- avg_constraints_per_step
- max_constraints_per_step
- num_cache_alg_steps
- num_free_layer_steps
- symmetry_depth
- num_symmetry_orientations
- num_edge_constraints
- num_corner_constraints
- num_orientation_constraints
- constraint_type_diversity

ML Method: Linear Regression

- Using the features extracted from each method, the score is predicted with **Linear Regression**
- Model uses weighted sum of features
  - $h(x) = \boldsymbol{\theta}_0 + \boldsymbol{\theta}_1 x_1 + \boldsymbol{\theta}_2 x_2 + \ldots + \boldsymbol{\theta}_n x_n$
- Learns weights using gradient descent
- Minimizes prediction error using cost function
  - $J(\theta) = \left(\frac{1}{2m}\right)\sum_{i=1}^{m}\left(h(x^i) - y^i\right)^2 + \left(\frac{\lambda}{2m}\right)\sum_{j=1}^{n}\theta_j^2$
- Each feature gets a weight, and model learns how important each is

ML Method: Random Forest

- Trains 200 decision trees:
  - Recommended 100–500 range before diminishing returns
- Trees grow fully unpruned, high individual variance, corrected by ensemble averaging
- Final prediction is the mean across all 200 trees (regression aggregation)
- Outputs feature_importances_ ranked by contribution, measures how much each feature reduces prediction error across all splits



Results

```
Mean absolute error: 0.000797
Mean absolute percentage error: 9.22%
Max over-prediction:        +0.003340
Max under-prediction:       -0.006893
```

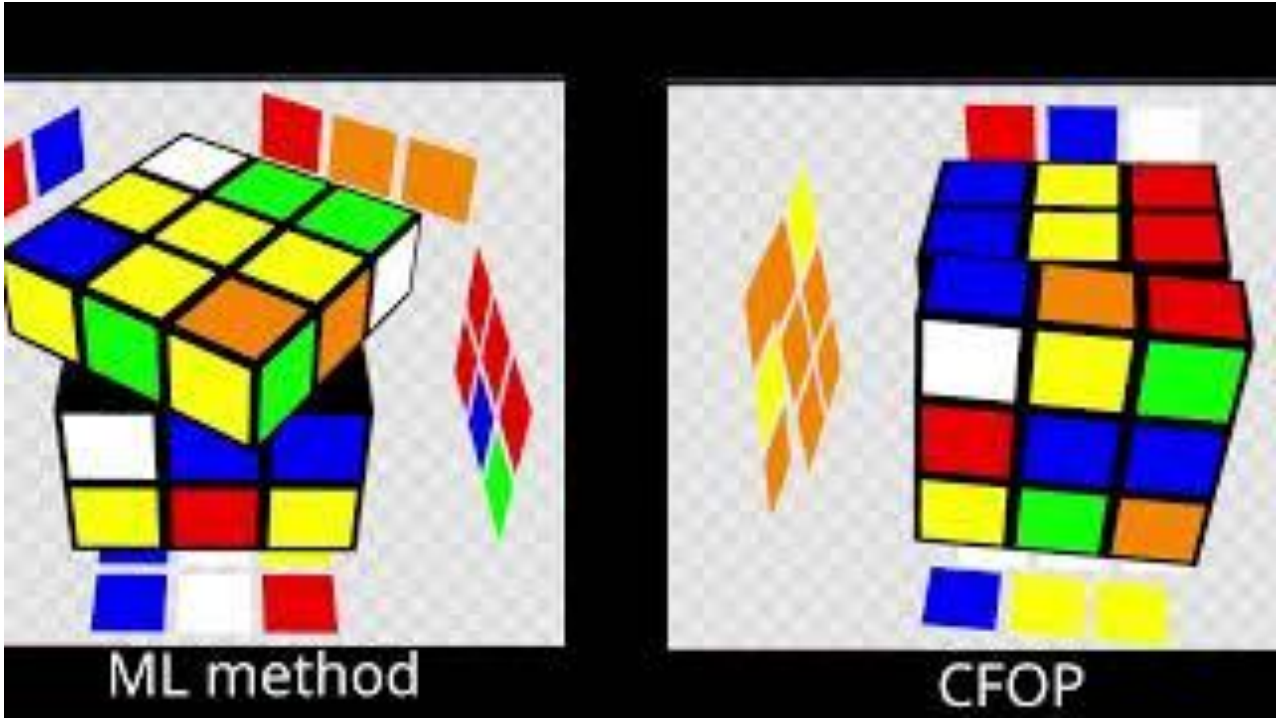
Random Forest

```
Mean absolute error: 0.001746
Mean absolute percentage error: 20.62%
Max over-prediction:        +0.006084
Max under-prediction:       -0.013534
```

Linear Regression

```
actual_rank,method,predicted_score,actual_score,avg_total_
1,hill_03cc015e,0.0210585450,0.025253,39.6,6,6.6,10,9
2,hill_45c24760,0.0210585450,0.024876,40.2,6,6.7,10,8
3,hill_ae832b91,0.0210604050,0.024691,40.5,6,6.75,10,7
4,hill_7c912a5e,0.0210604050,0.024691,40.5,6,6.75,10,7
5,hill_efa4c405,0.0210585450,0.024691,40.5,6,6.75,10,9
6,hill_de3d23c1,0.0210604050,0.02445,40.9,6,6.82,10,9
7,hill_e089562b,0.0210604050,0.02445,40.9,6,6.82,10,9
8,hill_5afebc0e,0.0210604050,0.024213,41.3,6,6.88,10,8
9,hill_f4becc7e,0.0210604050,0.024213,41.3,6,6.88,10,10
10,hill_75f90d8b,0.0210585800,0.024213,41.3,6,6.88,10,10
```

Demo video



- ML method: 45 moves
- CFOP: 50 moves

Limitations/Future Work

- Difficult to quantify some methods
- No turning heuristics for solutions - everything is move count based.
- If a “better” method is found, the community could reject it

Conclusions

Our oracle was successful in scoring methods accurately to judge a methods performance

Links

- Twizzle
- Repo
- kubesovler
- Cubing Club @ VT's Website